Projeto 3: Modelo generativo para o Fashion-MNIST com Autoencoders Variacionais convolucionais

Aline Breda Kuster (RA: 168495)
Introdução às Redes Neurais Artificiais, UNIFESP
Repositório: https://github.com/aline-kuster/Projeto-3---Redes-Neurais

1 Objetivo e dados

O objetivo do trabalho é treinar um modelo capaz de gerar imagens de roupas e entender como ele organiza essas imagens internamente. Para isso usei o Fashion-MNIST, um conjunto de 70 000 fotos de 28×28 pixels em tons de cinza, divididas em dez classes: camiseta, calça, pulôver, vestido, casaco, sandália, camisa, tênis, bolsa e bota.

Dos 60 000 exemplos de treino originais, separei 6 000 para validação (600 de cada classe, para manter o equilíbrio) e usei os 54 000 restantes no treino. A validação serviu para escolher o melhor modelo. Os 10 000 exemplos de teste ficaram guardados e só entraram na análise final. Os pixels foram divididos por 255 para ficar entre 0 e 1. Os rótulos das classes nunca participam do treino: o modelo vê apenas as imagens, e os rótulos só aparecem depois, para interpretar o resultado.

2 O modelo

Um autoencoder variacional (VAE) tem duas partes. O *encoder* recebe a imagem e a resume em poucos números, que formam o *espaço latente*. O *decoder* faz o caminho inverso e tenta reconstruir a imagem a partir desses números. Treinei duas versões, uma com latente de 2 dimensões e outra com 8.

A diferença para um autoencoder comum é que o encoder não devolve um ponto fixo. Ele devolve uma média $\boldsymbol{\mu}$ e uma variância $\boldsymbol{\sigma}^2$, e o código é sorteado dessa região: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, com $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Escrever o sorteio dessa forma, conhecida como truque da reparametrização, deixa o ruído isolado em $\boldsymbol{\varepsilon}$ e permite calcular o gradiente em relação a $\boldsymbol{\mu}$ e $\boldsymbol{\sigma}$.

A função de custo é o negativo da ELBO, calculada por imagem:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\sum_{i=1}^{784} \mathrm{BCE}(x_i, \hat{x}_i)}_{\text{reconstrução}} + \underbrace{\tfrac{1}{2} \sum_{j=1}^{d} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right)}_{\text{KL}}.$$

Os dois termos puxam em sentidos opostos. A reconstrução mede o quanto a imagem gerada se parece com a original e favorece códigos precisos e espalhados. O termo KL aproxima a distribuição de cada código de uma normal padrão, o que mantém o espaço latente contínuo e concentrado perto da origem. É por causa dele que dá para sortear um $\mathbf{z}$ qualquer da normal e obter uma roupa plausível. As perdas estão em nats por imagem.

Optei por uma arquitetura convolucional, porque as convoluções olham a imagem por regiões e aprendem padrões locais, como bordas, golas e solas, com bem menos pesos do que uma camada densa ligada aos 784 pixels. O encoder tem duas convoluções com passo 2, que reduzem a imagem de 28×28 para 14×14 e depois para 7×7, seguidas de uma camada densa e das saídas $\boldsymbol{\mu}$ e

$\log \boldsymbol{\sigma}^2$. O decoder espelha esse caminho com convoluções transpostas e termina numa sigmoide. Usei BatchNorm, ativação LeakyReLU e o otimizador Adam.

3 Escolha da topologia

O número de filtros, a largura da camada densa, a taxa de aprendizado e o tamanho do lote foram definidos por uma busca aleatória, feita separadamente para cada dimensão latente, já que a melhor arquitetura para 2 dimensões não precisa ser a mesma para 8. Os valores possíveis eram 16/32 ou 32/64 filtros, camada densa de 128, 256 ou 512 unidades, taxa de aprendizado de $5 \cdot 10^{-4}$, 10^{-3} ou $2 \cdot 10^{-3}$ e lote de 64 ou 128. Sorteei 8 combinações para cada dimensão e treinei cada uma por 10 épocas. A que teve a menor perda na validação foi treinada de novo, do zero, por até 60 épocas, com parada antecipada caso a validação ficasse 6 épocas sem melhorar.

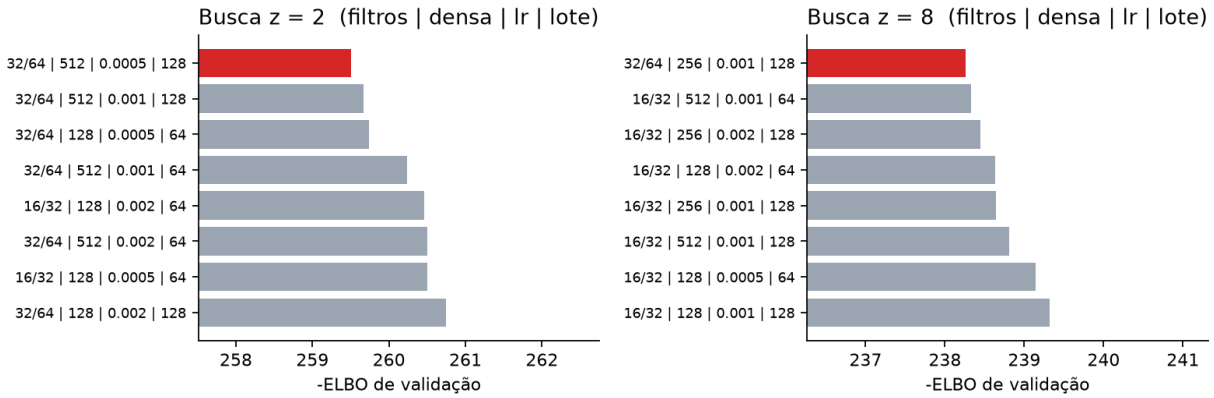


Figura 1: Perda de validação das combinações testadas (filtros | camada densa | taxa de aprendizado | lote). Em vermelho, a escolhida para cada dimensão latente.

A Figura 1 mostra que as combinações ficaram próximas, com cerca de 1 nat de diferença entre a melhor e a pior. A topologia, dentro desse intervalo, pesa pouco no resultado. Mesmo assim as escolhas foram diferentes: com 2 dimensões venceu a camada densa de 512 unidades e a menor taxa de aprendizado, e com 8 dimensões venceu uma camada densa de 256, com o dobro da taxa. O modelo de 8 dimensões terminou com metade dos parâmetros do outro (Tabela 1).

Tabela 1: Modelos finais. Perdas em nats por imagem. O PSNR mede a fidelidade da reconstrução (maior é melhor).

Latente	Filtros, densa	Parâmetros	Perda val.	Perda teste	Reconstrução	KL	PSNR (dB)
2 dim.	32/64, 512	3,29 M	255,91	258,00	251,55	6,44	16,60
8 dim.	32/64, 256	1,68 M	234,72	236,86	222,67	14,20	20,14

No treino final, as perdas de treino e validação andaram juntas o tempo todo, e a de teste ficou só 2 nats acima da de validação, então não houve sobreajuste. A melhor época foi a 56 no modelo de 2 dimensões e a 58 no de 8, das 60 permitidas, sinal de que a perda ainda caía devagar e mais épocas trariam algum ganho.

O modelo de 8 dimensões ficou 21 nats melhor. A reconstrução melhorou 29 nats e o KL subiu 8: com mais dimensões o encoder consegue guardar mais detalhes de cada peça e paga por isso no termo de regularização. A Figura 2 mostra a diferença na prática. Com 2 dimensões a reconstrução acerta o formato geral, mas vira uma versão genérica da classe, e a sandália de salto perde as

tiras. Com 8 dimensões as tiras e o salto aparecem. Nos dois casos as imagens saem um pouco borradas e a estampa da camiseta some, porque o decoder trata cada pixel de forma independente e acaba produzindo uma média das possibilidades.



Figura 2: Uma imagem de teste por classe (primeira linha) e as reconstruções dos dois modelos.

Reconstruir não é a mesma coisa que gerar. Para gerar roupas que não existem no conjunto de dados, sorteei pontos $\mathbf{z}$ da normal padrão e passei cada um pelo decoder, sem usar o encoder (Figura 3). As amostras do modelo de 2 dimensões são todas reconhecíveis, mas borradas e parecidas entre si. As do modelo de 8 dimensões têm mais detalhe e variedade, com solas, golas e recortes visíveis, embora algumas saiam fragmentadas. Em 8 dimensões os códigos das imagens de treino não preenchem toda a normal, e parte dos sorteios cai em regiões que o decoder quase não viu durante o treino.

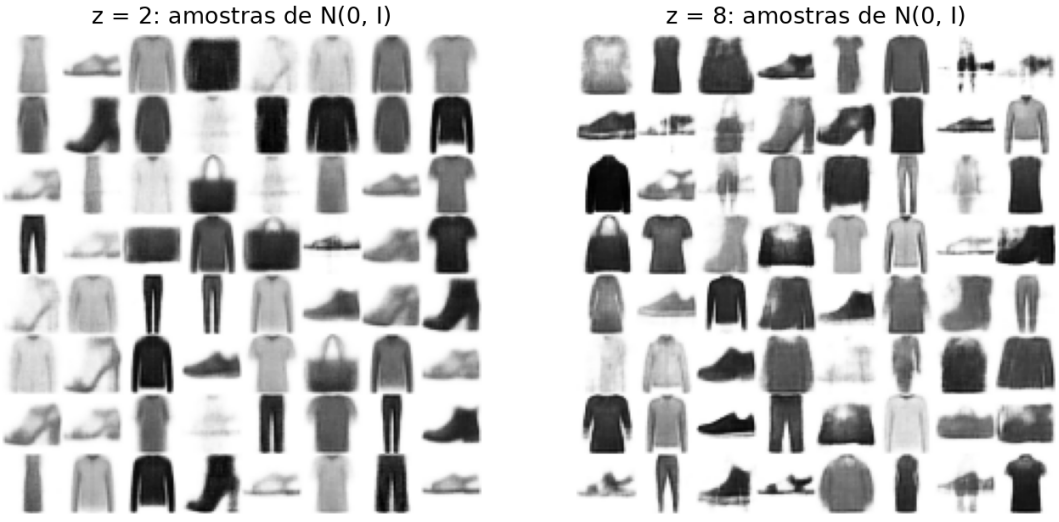


Figura 3: 64 roupas geradas por cada modelo a partir de pontos sorteados de $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

4 O espaço latente com 2 dimensões

Com apenas 2 dimensões, cada imagem vira um ponto num plano, e o espaço pode ser desenhado diretamente. Na Figura 4 cada ponto é a média $\boldsymbol{\mu}$ de uma imagem de teste. Usei a média, e não o $\mathbf{z}$ sorteado, para que o gráfico não dependa do ruído da amostragem. O painel da esquerda pinta os pontos pela classe, e o da direita mostra os mesmos pontos sem cor, só com a densidade.

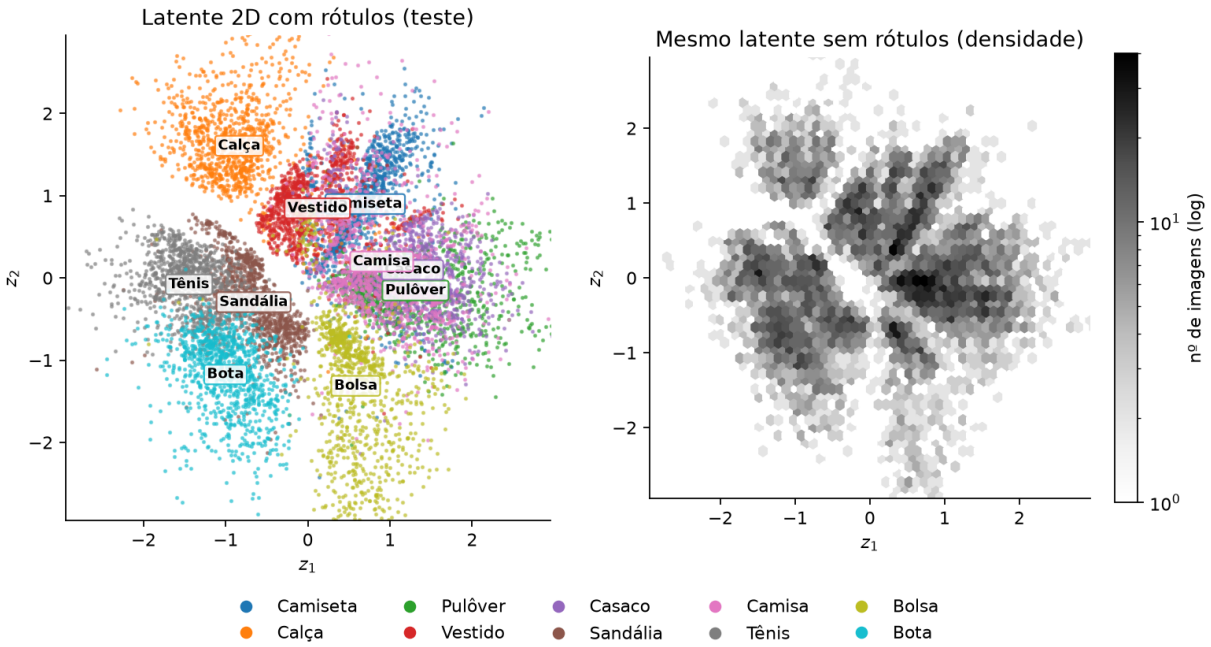


Figura 4: Espaço latente de 2 dimensões (conjunto de teste). À esquerda, com as classes. À direita, apenas a densidade de pontos.

As classes se organizam em fatias ao redor da origem. A calça forma um grupo isolado no alto à esquerda, os calçados ficam à esquerda (tênis em cima, sandália no meio e bota embaixo), a bolsa desce pelo centro e as roupas de cima ocupam o lado direito.

Há formação de clusters? Sim. Mesmo sem as cores, o painel da direita mostra quatro ou cinco regiões densas separadas por faixas quase vazias. Para medir isso sem usar os rótulos, apliquei o K-means com 2 a 14 grupos e calculei a silhueta, que vai de -1 a 1 e é maior quando os grupos estão bem separados. O melhor resultado foi 0,42, com 3 grupos. Esse número precisa de uma referência, porque o K-means sempre encontra algum agrupamento, mesmo em dados sem estrutura: aplicado a pontos sorteados de uma normal em 2D, ele dá silhueta de 0,32. O latente fica acima disso, então os grupos são reais. Eles correspondem a famílias de peças (calçados, roupas de cima, calças e bolsas) e não às dez classes.

Há separação dos rótulos? Em parte. Treinei um classificador linear (regressão logística) usando os códigos de treino e testei nos de teste. Ele acerta 67,2% das classes, contra 10% de um palpite aleatório, mesmo com o VAE nunca tendo visto os rótulos. A Figura 7 mostra onde estão os erros. Calça, bota, bolsa, sandália e tênis ficam acima de 80% de acerto. Pulôver, casaco e camisa ficam entre 31% e 42% e se confundem entre si, o que faz sentido: as três têm a mesma silhueta de manga longa e diferem em detalhes como botões e gola, que duas dimensões não conseguem guardar.

5 O espaço latente com 8 dimensões

Oito dimensões não podem ser desenhadas, então projetei os códigos em 2D com PCA. A PCA foi ajustada nos códigos de treino e aplicada aos de teste.

Quanto da variância a projeção ilustra? Apenas 27,8%: o primeiro componente explica 14,1% e o segundo, 13,7%. A Figura 5 mostra que todos os oito componentes explicam quase a mesma coisa, entre 10,7% e 14,1% cada. Isso é consequência direta do termo KL, que empurra os códigos para uma normal padrão, uma nuvem redonda com a mesma variância em todas as

direções. Como a PCA escolhe as direções de maior variância e nenhuma se destaca, qualquer par de componentes fica com pouco mais de um quarto da informação. Também vale lembrar que a PCA não usa os rótulos, então nada garante que as direções de maior variância sejam as que separam as classes.

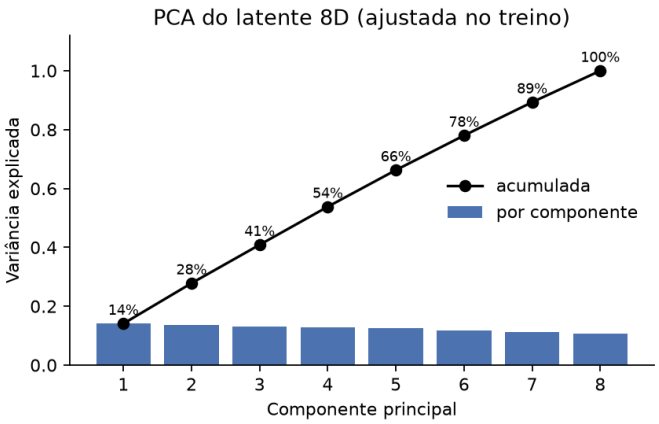


Figura 5: Variância explicada por componente principal (barras) e acumulada (linha) no latente de 8 dimensões.

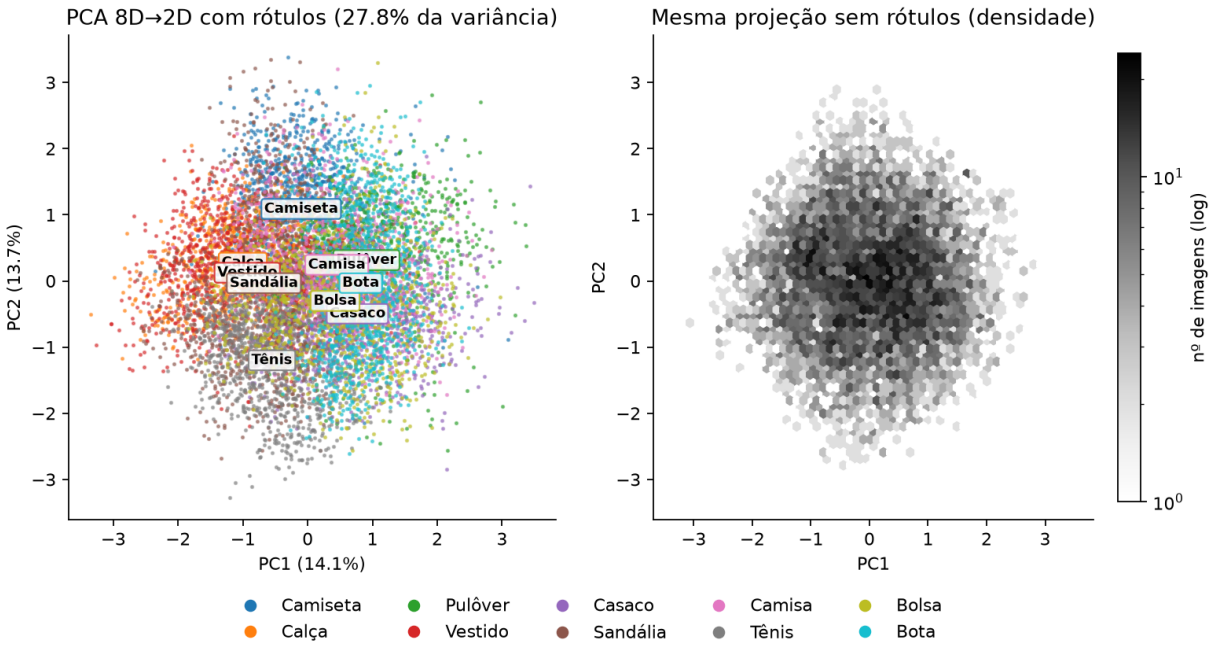


Figura 6: Projeção do latente de 8 dimensões nos dois primeiros componentes da PCA (conjunto de teste). À esquerda, com as classes. À direita, apenas a densidade.

Há formação de clusters? No espaço completo de 8 dimensões, sim, mas fracos. A silhueta do K-means chega a 0,13, o dobro dos 0,07 que pontos aleatórios em 8D produzem. Os grupos existem, só que são alongados e encostam uns nos outros. Na projeção da Figura 6 eles desaparecem: a densidade forma uma única mancha arredondada, e a silhueta do K-means no plano (0,34) é praticamente a mesma de pontos aleatórios (0,32).

Há separação dos rótulos? No espaço completo, mais do que com 2 dimensões. O mesmo classificador linear acerta 76,2% das classes, e pulôver e casaco sobem para 64% de acerto, embora a camisa continue em 40%. Na projeção, porém, as cores se misturam e o acerto cai para 30,7%.

Dá para notar algumas tendências na Figura 6 (vestido e calça à esquerda, tênis embaixo, camiseta em cima), mas os centros das classes ficam todos perto da origem. A informação de classe está no latente, espalhada pelas 8 dimensões, e o plano dos dois primeiros componentes descarta a maior parte dela.

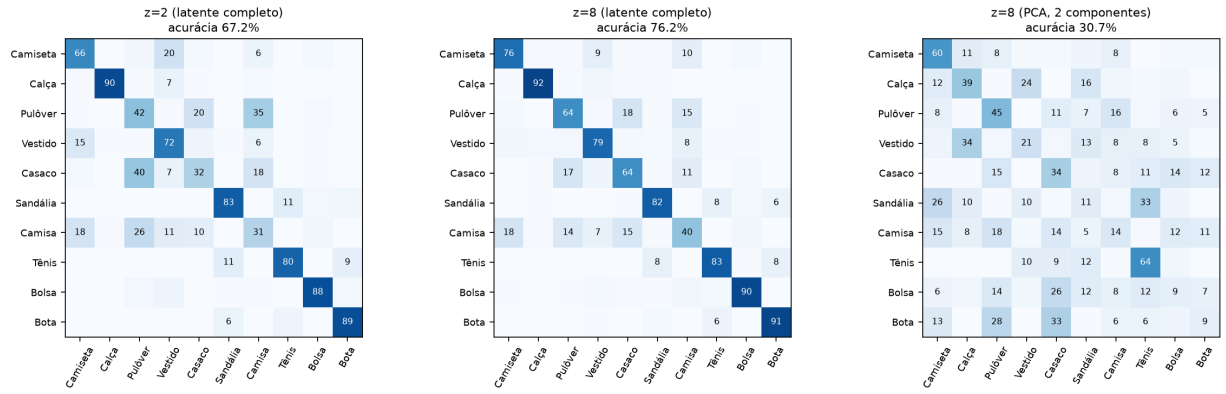


Figura 7: Matrizes de confusão do classificador linear no teste (% de cada classe real). A diagonal é o acerto.

Tabela 2: Resumo das medidas do espaço latente no conjunto de teste.

Representação	Silhueta K-means	Silhueta de pontos aleatórios	Acerto do classificador
2 dimensões	0,42	0,32	67,2%
8 dimensões	0,13	0,07	76,2%
8 dimensões, projeção PCA	0,34	0,32	30,7%

6 Considerações finais

Os dois modelos aprenderam a organizar as roupas sem nunca ver os rótulos, mas de formas diferentes. O de 8 dimensões reconstrói melhor e separa melhor as classes, enquanto o de 2 dimensões pode ser visto diretamente e mostra grupos claros, que correspondem a famílias de peças. A PCA se mostrou uma forma fraca de visualizar o latente de 8 dimensões: o termo KL deixa a nuvem de códigos redonda, os dois primeiros componentes guardam só 27,8% da variância e a estrutura de classes some na projeção. Em todos os casos, pulôver, casaco e camisa continuaram difíceis de separar. Como próximos passos, treinaria por mais épocas, já que a validação ainda melhorava na época 60, e testaria uma projeção não linear para o latente de 8 dimensões.